ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Bài 4 Functional Modeling
Mô hình hóa nghiệp vụ/chức năng

- hung.letan@hust.edu.vn
- 0913030731

ONE LOVE. ONE FUTURE.

Nội dung của bài học

- Hiểu bản chất và vai trò của Functional Modeling trong phân tích hệ thống thông tin.
- Phân biệt Business Process, System Function, Use Case, Activity và Feature.
- Phân tích AS-IS → Pain Points → Business Needs → TO-BE để xác định nhu cầu thay đổi của hệ thống.
- Thực hiện Functional Decomposition theo nguyên tắc top-down, abstraction và separation of concerns.
- Xây dựng Functional Model bằng các kỹ thuật phù hợp như Function Tree, Use Case, Scenario, Activity Diagram và DFD.
- Kiểm tra tính đầy đủ, nhất quán và truy xuất được giữa Business Needs, Requirements và System Functions.

Tại sao cần Functional Modeling ?

- Requirement modeling
  - -> system cần đạt được gì thông qua: Goals, Needs và Requirements.
- Tuy nhiên,
  - yêu cầu vẫn chưa cho biết hệ thống phải thực hiện những hoạt động nào để đáp ứng yêu cầu đó.
  - Chuyển requirement-oriented thinking sang function-oriented thinking.
    - từ **"Khách hàng nói họ muốn gì"** sang **"Hệ thống cần làm gì để giải quyết vấn đề"**.
- Việc chuyển gọi là Functional Modeling
  - System cần thực hiện gì?
    - -> xác định, tổ chức và phân rã những gì hệ thống phải thực hiện.

Chức năng hệ thống (phần mềm) ?

- Chức năng của hệ thống
  - khả năng hoặc hoạt động mà hệ thống thực hiện để tạo ra kết quả có ý nghĩa
  - đáp ứng của hệ thống với các nhu cầu của người dùng trong nhiệm vụ, hành động, hoạt động nghiệp vụ
- Vi dụ: Chức năng hệ thống quản lý đào tạo
  - 1. Quản lý nội dung và khóa học (Course Management)
  - 2. Quản lý người dùng (User Management)
  - 3. Tương tác và thảo luận (Interaction Tools)
  - 4. Kiểm tra và đánh giá (Assessment & Testing)
  - 5. Theo dõi tiến độ và báo cáo (Tracking & Reporting)
  - 6. Cấp chứng nhận (Certification)

Tại sao Hệ thống có chức năng ?

- Cty hoạt động nhờ các chức năng kinh doanh (Business Funtions),
  - Ví dụ
    - Các chức năng kinh doanh: Bán hàng, Nhân sự.
- Để hỗ trợ chức năng kinh doanh, hệ thống cần quy trình số cụ thể trong phần mềm hỗ trợ các chức năng kinh doanh
  - Đảm bảo công nghệ giải quyết được các nhu cầu thực tế
  - Vd: Các chức năng hệ thống:“Tạo hóa đơn" , "Cập nhật hồ sơ nhân viên") .
- Đây gọi là chức năng hệ thống
  - Thu hẹp khoảng cách giữa hoạt động doanh nghiệp và thiết kế kỹ thuật.
- Chuyển đổi từ chức năng kinh doanh sang các chức năng hệ thống
  - thể hiện sự thay đổi từ "những gì công ty làm" sang "cách công nghệ tự động hóa nhiệm vụ đó".

Những điểm khác biệt chính

- Chức năng kinh doanh:
  - xác định bởi lĩnh vực kinh doanh, ổn định theo thời gian và mô tả "những gì" đang được thực hiện
  - Thuộc sở hữu và được thực hiện bởi các nhóm, phòng ban hoặc vai trò.
  - Ví dụ
    - xử lý bảng lương , quản lý kho hàng và quy trình tiếp nhận khách hàng
- Chức năng hệ thống:
  - Được định nghĩa thông qua logic kỹ thuật, câu chuyện, yêu cầu người dùng,
  - Nó mô tả "cách thức" hệ thống thực hiện một tác vụ
  - Có thể thay đổi tùy thuộc vào hạn chế kỹ thuật hoặc nâng cấp hệ thống.
  - ví dụ:
    - "Tính toán khấu trừ thuế" , "Tạo hóa đơn, chuyển khoản trực tiếp"

Nhầm lẫn lớn nhất: Business Process => System Function

- Business Process – Đặt lịch khám

  Khách hàng yêu cầu khám

  ↓

  Nhân viên kiểm tra lịch bác sĩ

  ↓

  Chọn thời gian

  ↓

  Nhập thông tin

  ↓

  Xác nhận lịch

  ↓

  Thông báo khách hàng

- chuyển hoạt động thành Function một cách máy móc.
- System Functions
  - Receive Request
  - Check Schedule
  - Select Time
  - Enter Information
  - Confirm
  - Notify
- Business Process không được chuyển trực tiếp thành System Function.

Trách nhiệm của Business Activity

- Business Activity
  - bước hoặc nhiệm vụ trong một quy trình kinh doanh, giúp chuyển đổi đầu vào thành đầu ra mong muốn.
- Trong Business process:
  - Activity nào hệ thống phải chịu trách nhiệm thực hiện ? => System funtion

Business Activity	**Responsibility**	**Có phải System Function?**
Tiếp nhận yêu cầu	Nhân viên	Có thể không
Kiểm tra lịch	System	✓
Chọn thời gian	Customer + System	✓ System support
Nhập thông tin	Customer	Không nhất thiết
Xác nhận lịch	System và Staff	✓ tùy TO-BE
Khám bệnh	Bác sĩ	✗
Gửi SMS	System	✓

Nhầm Business Function = System Function

- Business Function
  - hoạt động cốt lõi của tổ chức từ góc độ quản lý
- System Function
  - tính năng kỹ thuật do hệ thống CNTT thực hiện hỗ trợ các nghiệp vụ đó
- Không thể chuyển 1-1 mà phải qua các bước
  - Decomposition -Phân rã
  - Automation mapping
  - Hành động của hệ thống

- Ví dụ
- Business Function

Clinical Services

↓

Business Processes

Appointment Management
Examination
Treatment
Follow-up

↓

System Functions

– Search Doctor
– Check Availability
– Book Appointment
– Record Examination
– Create Treatment Plan
– Schedule Follow-up

Tổng hợp Những nhầm lẫn phổ biến nhất

Business Process	Không hoàn toàn	Process mô tả công việc diễn ra như thế nào; Function mô tả hệ thống phải làm gì
Use Case	Gần, không đồng nhất	Use Case mô tả một mục tiêu tương tác Actor–System; Function là trách nhiệm/chức năng hệ thống
Activity	Không	Activity là một bước/hành động trong flow; Function có thể bao gồm nhiều activity
Button/Menu	Sai	Button là UI control; chỉ là cách kích hoạt một chức năng
Requirement	Sai	Requirement nói hệ thống cần đạt điều gì; Function cụ thể hóa hệ thống thực hiện việc gì
Business Function	Sai cấp độ	Business Function thuộc tổ chức; System Function thuộc hệ thống
Mỗi bước BPMN	Sai	Có bước do người thực hiện, có bước do hệ thống, có bước do hệ thống bên ngoài

Mô hình hóa chức năng

- Functional modeling
  - Chuyển đổi các nhu cầu chức năng kinh doanh trừu tượng thành mô tả cụ thể có cấu trúc về những gì hệ thống phải làm được
  - không đơn giản là liệt kê các chức năng trên menu của phần mềm.
- Mô hình chức năng phải cho thấy
  - chức năng được tổ chức thế nào,
  - có quan hệ gì với nhau,
  - nhận input gì, tạo output gì và
  - góp phần đáp ứng requirement nào.
- Vì vậy - đây là một hoạt động phân tích,
  - không phải chỉ là hoạt động vẽ UML.

“good function”

- Một “good function”
  - Goal rõ.
  - Input rõ.
  - Output rõ.
  - Responsibility rõ.
  - Behavior có thể mô tả.
- Một hệ thống doanh nghiệp có thể chứa hàng trăm chức năng.
  - Nếu xem toàn bộ system như một khối duy nhất, việc phân tích sẽ quá phức tạp.
  - Nếu chia qua chi tiết dẫn đến dẫn đến
    - độ phức tạp về mặt quản lý,
    - rủi ro phân mảnh dữ liệu và
    - chi phí tích hợp tăng cao.

High abstraction

↓

Appointment Management

↓

Book Appointment

↓

Check Availability

↓

Query Appointment Database

↓

Low abstraction

Functional Decomposition

Phân rã Decomposition

- Functional Decomposition
  - kỹ thuật phân rã Chức năng ở mức cao thành các các chức năng con ở mức thấp hơn,
  - cho đến khi mỗi chức năng đủ rõ để có thể phân tích, mô tả và thiết kế.
- Mục đích:
  - Giảm độ phức tạp.
  - Tạo cấu trúc phân cấp.
  - Làm rõ phạm vi của từng chức năng.
  - Chuẩn bị cho Behavioral và Structural Modeling.
- Phân rã tốt phải dựa trên nguyên lý giúp quyết định
  - chia ở đâu, chia như thế nào và khi nào dừng.

Nguyên tắc phân rã Decomposition

- 1. Abstraction
  - Mỗi level thể hiện hệ thống ở một mức chi tiết khác nhau, có mức abstraction tương đối đồng nhất
- 2. Separation of Concerns
  - Mỗi function/sub-function nên tập trung vào một concern tương đối rõ.
- 3. Decomposition / Divide and Conquer
  - Thay vì xử lý một problem lớn chia thành các problem nhỏ hơn có thể hiểu và quản lý.
- 4. Cohesion
  - Các chức năng nằm cùng một nhóm nên có mục đích liên quan chặt chẽ.
- 5. Coupling
  - Các nhóm chức năng không nên phụ thuộc lẫn nhau một cách không cần thiết.

Làm thế nào biết đã phân rã "đủ"?

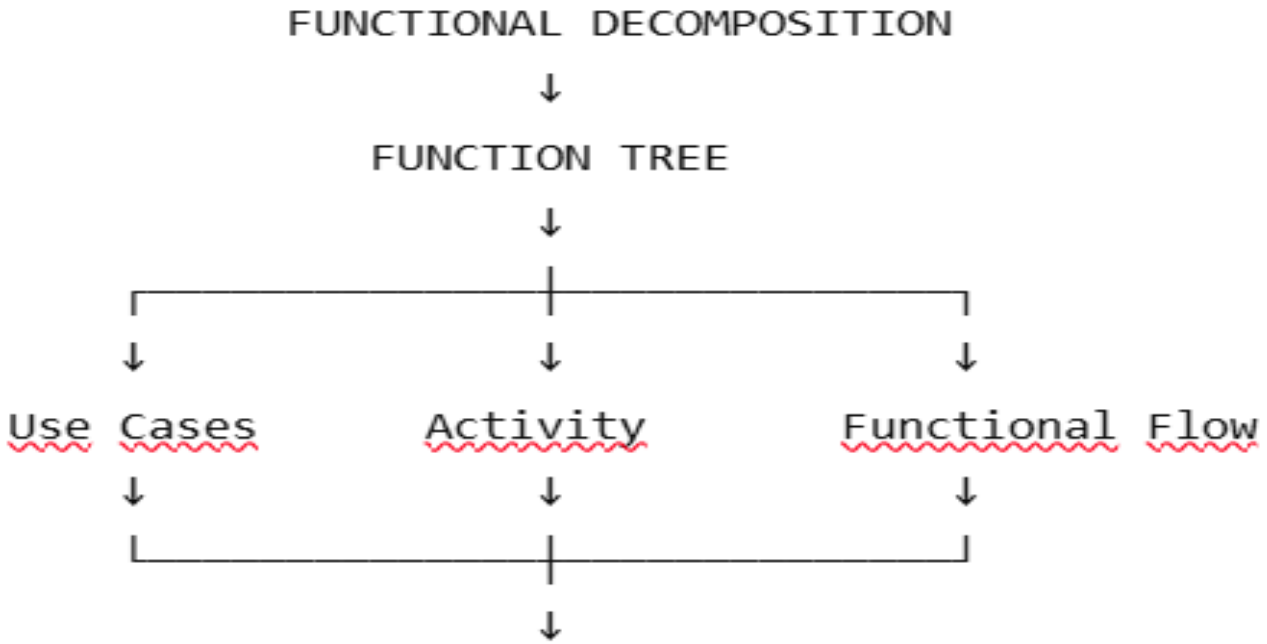
  - Nếu dừng quá sớm, function còn mơ hồ;
  - nếu phân rã quá sâu, mô hình trở thành danh sách các thao tác kỹ thuật.
- Nguyên tắc dừng khi function:
  - Có mục đích rõ ràng.
  - Có trách nhiệm tương đối độc lập.
  - Có thể mô tả được behavior.
  - Có thể xác định Actor hoặc trigger phù hợp.
  - Có thể xác định input/output chính.
  - Không cần tiếp tục phân rã để hiểu yêu cầu nghiệp vụ.

Functional Hierarchy / Function Tree _ Kết quả phân rã

- Function Tree hoặc Functional Hierarchy
  - công cụ trực quan đơn giản và hiệu quả để thấy toàn bộ chức năng của hệ thống trước khi đi sâu vào từng chức năng.
  - cho thấy cấu trúc phân cấp của các chức năng
- Function Tree trả lời:
  - “Hệ thống gồm những nhóm chức năng nào và
  - mỗi nhóm được phân rã ra sao?”
- Function hierarchy giúp:
  - Xác định scope.
  - Nhận diện missing functions.
  - Nhận diện duplicate functions.
  - Chuẩn bị Activity/DFD.
  - Chuẩn bị phân chia module.

Functional Decomposition → Functional Model

- Phân rã chức năng không phải đích cuối cùng.
  - Nó tạo ra **cấu trúc chức năng ban đầu**,
  - chức năng phải được kiểm chứng bằng yêu cầu, quy trình, dữ liệu và hành vi.
- Đầu ra Function Tree cần kết hợp với các kỹ thuật Mô hình chức năng khác để làm rõ các chức năng.
  - Use case
  - Activity
  - Flow

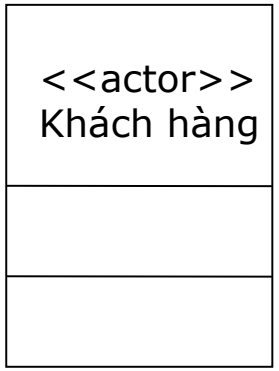


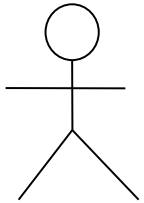
User case

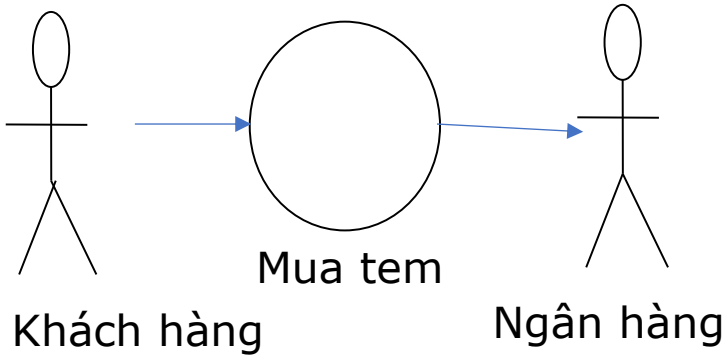
- User case
  - kỹ thuật để nắm bắt và thể hiện các yêu cầu chức năng của hệ thống bằng cách mô tả cách người dùng tương tác với hệ thống đó để đạt được các mục tiêu cụ thể.
- Trong Mô hình hóa yêu cầu,
  - Use Case được sử dụng để mô tả mục tiêu của actor khi tương tác với hệ thống
  - Mô tả mục tiêu không cần đi sâu vào chi tiết hoạt động bên trong của hệ thống
- Khi sang Functional Modeling,
  - chúng ta đi sâu hơn vào scenario của Use Case để nhận diện các system functions cần thực hiện.

Các yếu tố trong biểu đồ ca sử dụng

- Tác nhân (actor):
  - Người dùng, hệ thống bên ngoài có tương tác trực tiếp với hệ thống
  - Phân loại:
    - Đối tác chính: chủ động
    - Đối tác phụ: bị động
- Ca sử dụng (usecase):
  - Biểu diễn hành động, nhiệm vụ hoặc dịch vụ mà hệ thống thực hiện để mang lại giá trị cho tác nhân
  - hướng đến mục tiêu của người hành động.
  - Chúng độc lập với quá trình thực hiện.

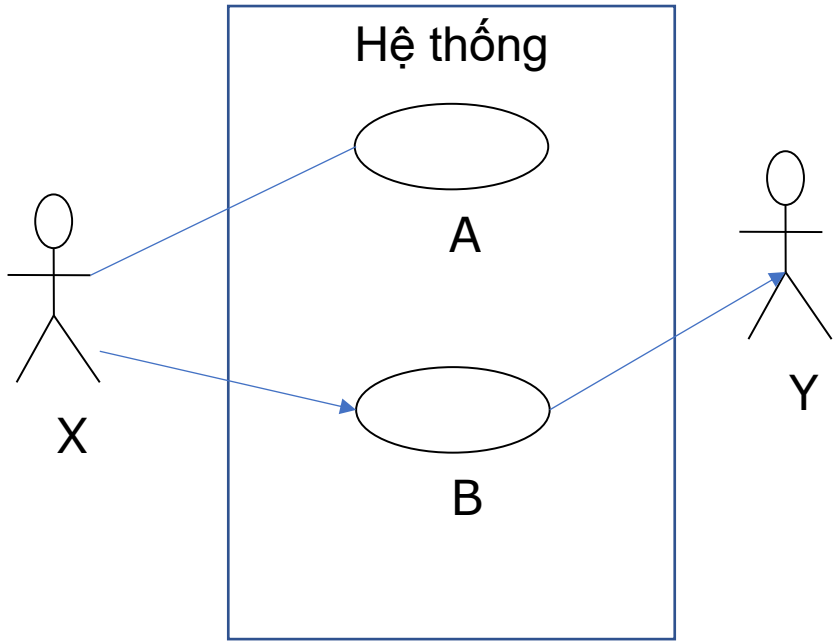






System Boundary

- Không thể xác định chức năng của system nếu chưa biết system nằm ở đâu trong môi trường.
  - Một hoạt động có thể là trách nhiệm của system, của con người hoặc của một external system.
- Câu hỏi:
  - System chịu trách nhiệm gì?
  - Actor nào bên ngoài?
  - External system nào tương tác?
  - Information nào đi qua boundary?

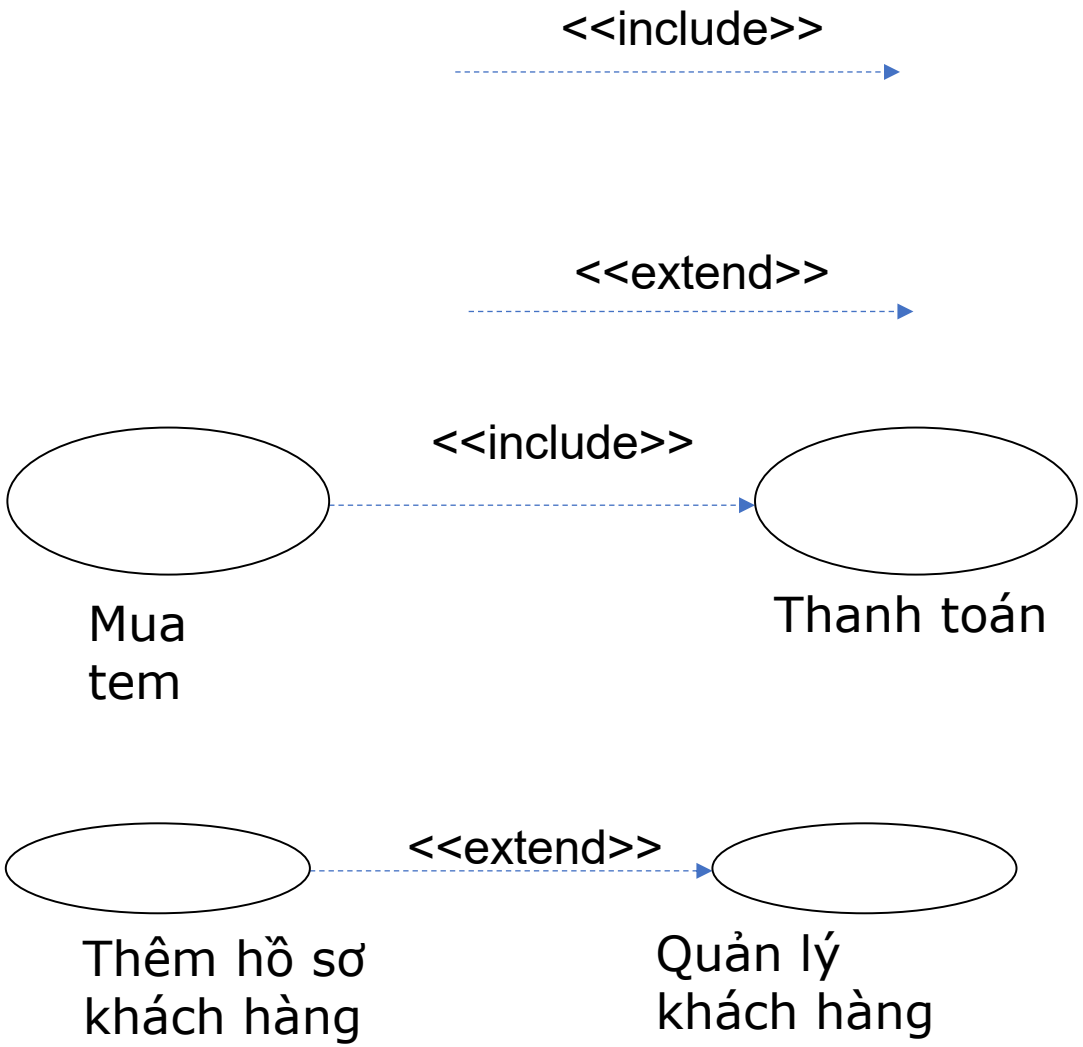


Boundary Identification -Context Modeling

- Context Modeling :
  - "Hệ thống này nằm ở đâu trong thế giới thực, nó tương tác với những ai/hệ thống nào bên ngoài, và phạm vi giải quyết của nó đến đâu?"
- Boundary xác định chức năng thuộc hệ thống hay môi trường?
  - Bên trong ranh giới là phần có thể thay đổi
  - Bên ngoài là phần không thể thay đổi (API thanh toán của bên thứ ba, quy định hiện hành của công ty, người dùng)
- Context Modeling và Boundary Identification phải xuất hiện trước hoặc song song với Functional Modeling.

Relationship (Mối quan hệ) giữa các ca sử dụng:

- Communication Link (Association):
  - thể hiện sự tương tác qua lại giữa Actor và Use Case .
- Relationship (Mối quan hệ):
  - Include (Bao gồm):
    - Use Case này bắt buộc phải chạy Use Case kia.
  - Extend (Mở rộng):
    - Use Case phụ có thể chèn thêm hành vi.
  - Generalization (Tổng quát hóa):
    - Quan hệ kế thừa giữa
- Mục đích:
  - Phân tách các ca sử dụng phức tạp thành các ca sử dụng đơn giản
  - Tránh sự trùng lặp khi phân tích, thiết kế



Use Cases → Function

- Use Case để nhận diện các system functions
  - Ánh xạ các mục tiêu trừu tượng của người dùng thành các hành vi cụ thể của hệ thống thông qua một kịch bản sử dụng.
  - Phân tích lộ trình cụ thể, từng bước một để thực ca sử dụng đó phân tích từ tổng quan hành vi người dùng đến chi tiết kỹ thuật
  - Tổng hợp các hành động do hệ thống (System) thực hiện trong kịch bản thành các chức năng (Function)
- Use Case là nguồn đầu vào quan trọng để nhận diện function,
  - nhưng không phải mapping 1–1.

Scenario luồng thực thi cụ thể của một Use Case.

- Scenario
  - Nó chuyển đổi một mục tiêu trừu tượng (User Goal) của người dùng thành một chuỗi các hoạt động chi tiết
- Scenario giúp xác định:
  - Functions
  - Sequence
  - Conditions
  - Alternative paths
  - System responses
- Functions (Chức năng):
  - Ví dụ:
    - Nút "Thanh toán",
    - ô "Nhập mã giảm giá",
    - tính năng "Gửi email xác nhận".

Scenario biến Use Case từ mục tiêu thành chuỗi hoạt động

- Sequence (Trình tự):
  - Sắp xếp các bước thực hiện theo một thứ tự logic từ bắt đầu đến khi kết thúc
    - (Bước 1: Nhập thông tin -> Bước 2: Nhấn xác nhận -> Bước 3: Hệ thống xử lý).
- Conditions (Điều kiện):
  - Xác định điều kiện tiên quyết trước và kết quả sau khi kịch bản hoàn thành
- Alternative paths (Luồng thay thế/Ngoại lệ):
  - Ví dụ:
  - Người dùng nhập sai mật khẩu, hệ thống mất kết nối mạng, hết hàng trong giỏ).
- System responses (Phản hồi của hệ thống):
  - (Ví dụ: Hiển thị thông báo lỗi, chuyển hướng trang, hiển thị pop-up thành công).

Các bước Use case giúp nhận diện function.

- Ví dụ: Book Appointment

```
Actor
  ↓
Goal
  ↓
Use Case
  ↓
Scenario
  ↓
System Actions
  ↓
Functions
```

Customer

↓

Book Appointment

↓

1. Search Doctor
2. Select Date
3. Check Availability
4. Select Slot
5. Enter Information
6. Confirm Booking
7. Create Appointment
8. Send Confirmation

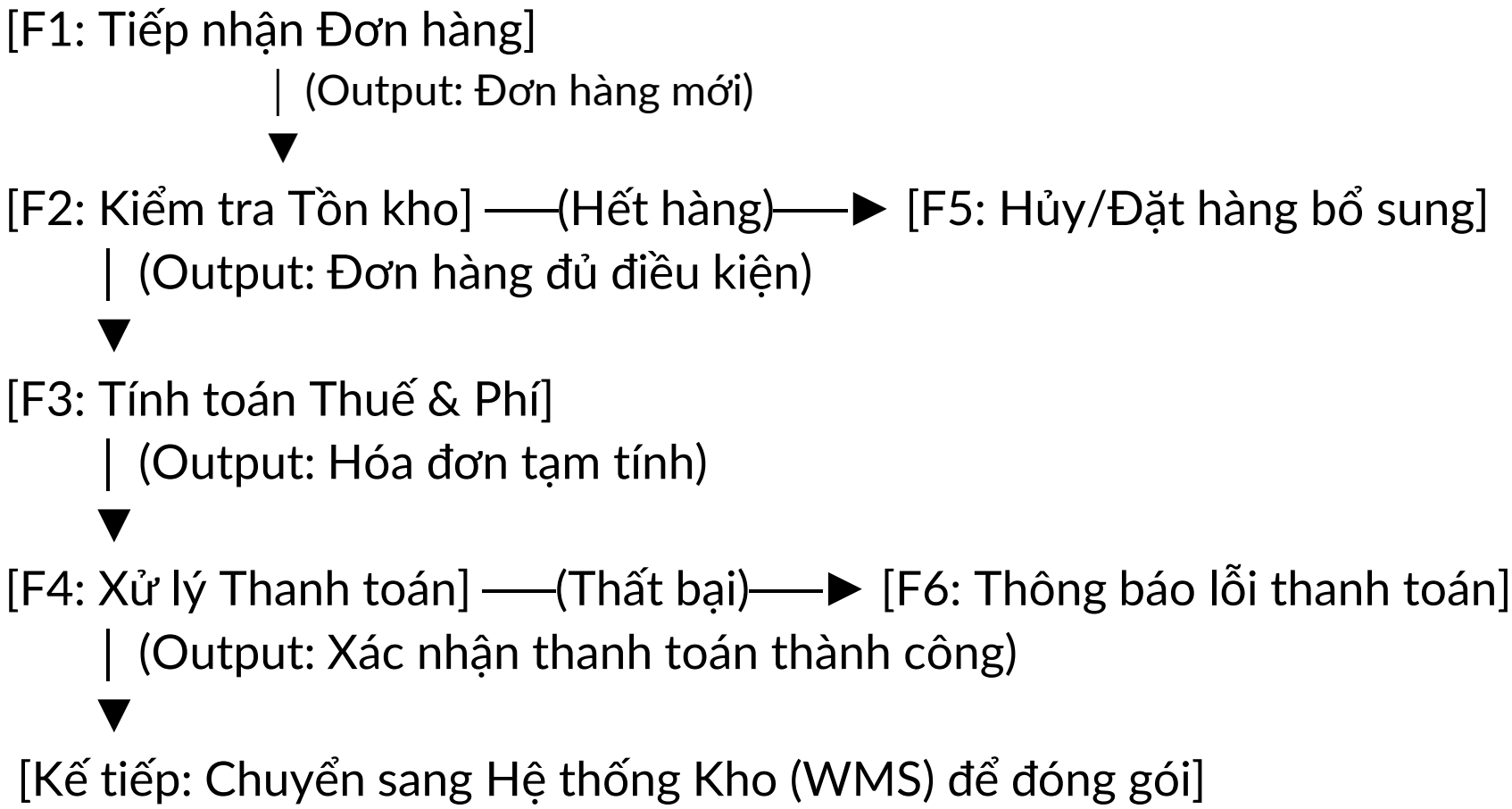
1. Chức năng Hệ thống (User-Facing Functions)
   - Tìm kiếm bác sĩ (Doctor Search):
   - Chọn ngày khám (Date Selection): Giao diện lịch (Calendar)
   - Chọn khung giờ (Slot Selection):
   - Nhập thông tin bệnh nhân (Patient Information Entry.
   - Xác nhận đặt lịch (Booking Confirmation Interface):
2. Tác vụ Hệ thống (System Functions)
   - Kiểm tra trạng thái trống (Check Availability):.
   - Khởi tạo cuộc hẹn (Create Appointment):
   - Gửi thông báo xác nhận (Send Confirmation):

Mô hình dòng chảy chức năng

- Functional decomposition cho biết “có những function nào”, cấu trúc nhưng chưa cho biết chúng liên hệ với nhau ra sao

- Functional Flow Modeling
  - kỹ thuật lập sơ đồ cách chuyển đổi đầu vào thành đầu ra thông qua chuỗi các chức năng và chuyển động dữ liệu theo trình tự
  - bổ sung liên kết bằng input, transformation, output và dependency giữa các chức năng.
- Nhờ các chiều liên kết này,
  - Functional Flow giúp thấy rõ luồng đi của dữ liệu/quy trình và phát hiện sớm các điểm nghẽn hoặc sai sót về mặt logic.
  - Functional Flow cho biết quan hệ vận hành giữa các chức năng.

Ví dụ: Sơ đồ chuỗi vận hành (Functional Flow)



Tại sao cách tiếp cận này quan trọng

- Tính toàn vẹn của dữ liệu:
  - Nếu chức năng F3 không nhận được Địa chỉ giao hàng từ F1 (truyền qua F2), nó sẽ không thể tính được phí ship và thuế, khiến hệ thống bị treo.
- Xử lý ngoại lệ (Exception Handling):
  - Nếu F2 báo hết hàng, luồng vận hành sẽ rẽ nhánh sang F5 thay vì đi tiếp đến F3. Mô hình này ép người thiết kế phải bao quát mọi kịch bản vận hành thực tế.
- Functional Flow đặc biệt hữu ích khi:
  - Một function gọi function khác.
  - Function có nhiều điều kiện.
  - Cần xác định dependency.
  - Chuẩn bị Behavioral Modeling.
  - Phân tích processing logic.

mô hình IPO-D (Input - Transformation - Output - Dependency)

- IPO-D
  - khung phân tích lập bản đồ quy trình kinh doanh theo bốn trụ cột cốt lõi
  1. Input (Đầu vào): "Chức năng này cần gì để bắt đầu?".
     - Dữ liệu nhập, file tải lên, dữ liệu từ Database, tín hiệu từ hệ thống khác (API).
  2. Transformation (Biến đổi): "Hệ thống làm gì với đầu vào?".
     - Thuật toán tính toán, quy tắc kinh doanh (Business Rules), kiểm tra điều kiện (Validation), mã hóa dữ liệu.
  3. Output (Đầu ra): "Kết quả trả về là gì?".
     - Thông báo trên màn hình, cập nhật trạng thái trong Database, sinh file (PDF/Excel), hoặc gửi email/Notification.
  4. Dependency (Sự phụ thuộc):
     - "Chức năng này liên kết với các chức năng khác như thế nào?".

Ví dụ thực tế: Tính năng "Thanh toán đơn hàng" (Checkout)

- 4 yếu tố phối hợp trong luồng chức năng thanh toán:
  - Input:
    - Mã đơn hàng (Order ID).
    - Phương thức thanh toán (Thẻ tín dụng, Ví điện tử...).
    - Số tiền cần thanh toán.
  - Transformation:
    - Hệ thống kiểm tra số dư hoặc tính hợp lệ của thẻ.
    - Tính toán lại thuế hoặc áp dụng mã giảm giá (nếu có).
    - Gọi API sang cổng thanh toán bên thứ ba (ví dụ: Stripe, MoMo) để trừ tiền.
  - Output:
    - Trạng thái đơn hàng chuyển thành "Đã thanh toán".
    - Hiển thị màn hình "Thành công" cho khách hàng.
    - Gửi email hóa đơn điện tử.
  - Dependency:
    - Phụ thuộc trước: Cần Output của "Tạo đơn hàng" làm Input cho chức năng Checkout).
    - Phụ thuộc sau: Output của (Trạng thái Đã thanh toán) sẽ kích hoạt làm Input cho "Xử lý vận chuyển / Giao hàng".

Phụ thuộc chức năng

- Functional Modeling trả lời
  - không chỉ “Cái gì?” mà còn “phụ thuộc vào cái gì?”
  - Sau khi xác định các function, cần biết function nào phụ thuộc function nào.
  - Một function có thể cần kết quả của function khác trước khi thực hiện.
- Phân tích dependency
  - giúp phát hiện thứ tự logic, coupling và các điểm có thể ảnh hưởng khi thay đổi requirement.
- Công việc Phân tích :
  - Xác định execution order.
  - Nhận diện coupling.
  - Chuẩn bị sequence/behavior model.
  - Phân tích impact khi thay đổi.

Activity Modeling

Tại sao cần Mô hình hóa hoạt động

- Phân rã chức năng
  - Các định các chức năng và Tổ chức các tầng chức năng
  - Xác định phạm vi xử lý
    - Cho cái nhìn tổng quan về những gì hệ thống phải thực hiện
  - Xác định các điểm đầu vào/đầu ra dữ liệu
    - làm rõ hệ thống cần nhận vào thông tin gì và trả ra kết quả gì.
- Câu hỏi tiếp: Các chức năng này được thực hiện theo trình tự nào?
  - Cần Activity Modeling.
    - Thể hiện luồng quy trình (Workflow)

Mô hình hóa hoạt động

- Activity Modeling
  - Biểu diễn quy trình làm việc từng bước của các quy trình kinh doanh, logic vận hành hoặc các ca sử dụng phức tạp
- Các thành phần cốt lõi của mô hình hoạt động:
  - Chuỗi hoạt động (Activities)
  - Quyết định (Decision)
  - Điều kiện (Condition)
  - Song song (Parallelism)
  - Dòng công việc (Workflow)

Activity Modeling và Functional Flow Modeling

- Khi functional flow trở nên phức tạp, mô tả bằng văn bản không còn đủ trực quan.
- Activity Diagram cho phép biểu diễn flow, decision, parallel activities và synchronization.
- Đây là một trong những kỹ thuật UML quan trọng nhất khi mô hình hóa chức năng và workflow.

- Khác biệt
  - Functional Flow tập trung vào quan hệ giữa functions;
  - Activity tập trung vào workflow và control flow.
    - Activity Model trả lời: **"Function được thực hiện theo flow nào?"**

Functional Flow vs Activity Modeling

- **Functional Flow tập trung:**

  Function
  ↓
  Function
  ↓
  Function

- Ví dụ:

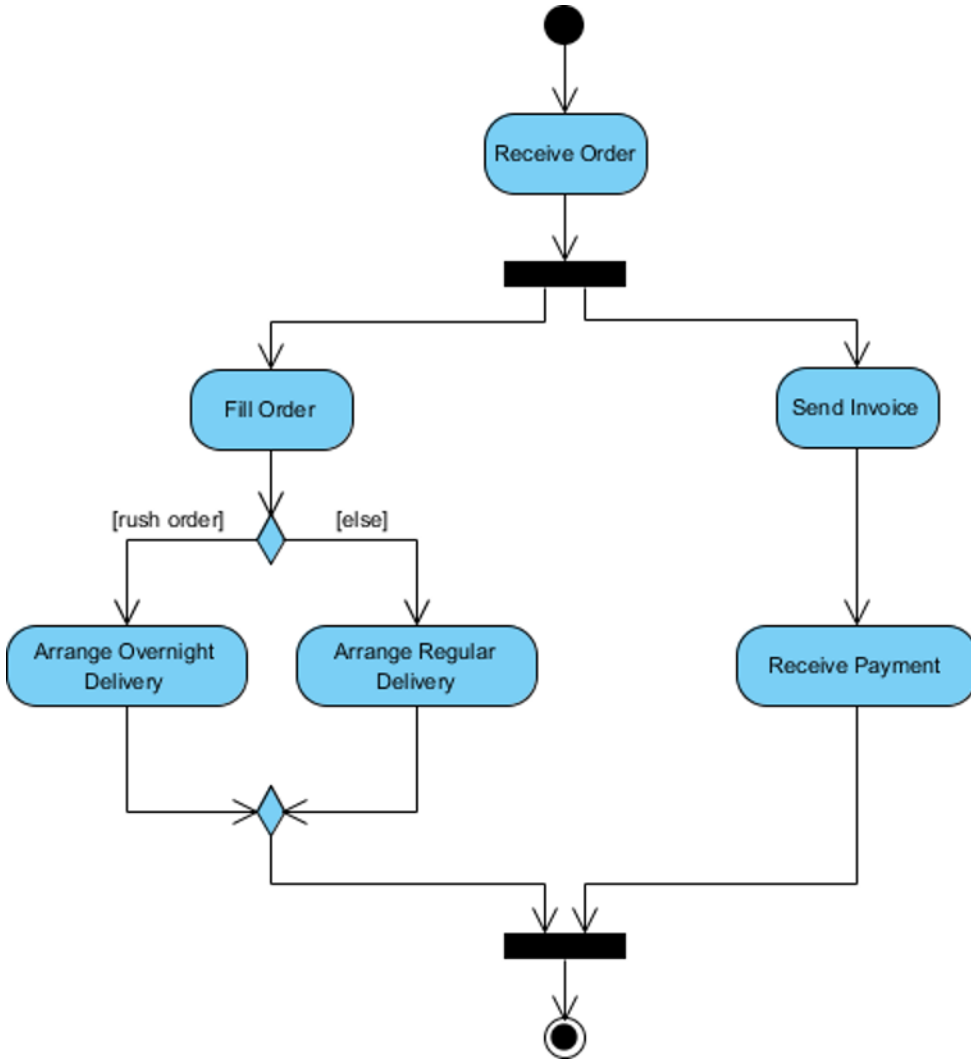
  Search Doctor
  ↓
  Check Availability
  ↓
  Book Appointment
  ↓
  Notify Customer

- **Activity Modeling tập trung:**
  - Action, Decision, Parallelism
  - Condition, Workflow
- Ví dụ:

  Search Doctor
  ↓
  Check Availability
  ↓
  [Available?]
  ↙ ↘
  No Yes
  ↓ ↓
  Suggest Date | Book Appointment

Activity diagram - UML

- activity diagram
  - biểu đồ hành động được sử dụng để mô hình hóa điều khiển luồng, dữ liệu luồng và quy trình dịch vụ từ bước này sang bước khác.
- Trong (UML),
  - được thiết kế đặc biệt để xử lý
    - logic phức tạp, các tác vụ song song và
    - vai trò trách nhiệm người tham gia
  - có thể gắn liền với các yếu tố khác như
    - người chịu trách nhiệm (WHO),
    - vị trí (WHERE),
    - thời gian (WHEN),
    - đối tượng dữ liệu (WHAT)



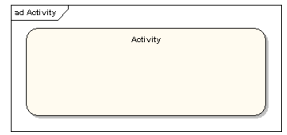
Vai trò của mô hình hoạt động trong OOAD

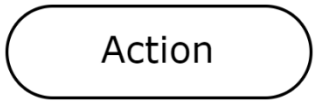
- Mô tả Use Case (Use Case realization):
  - Minh họa chi tiết các kịch bản phức tạp hoặc luồng sự kiện bên trong một Use Case.
- Quản lý song song (Parallel Processing):
  - Dùng các thanh Fork (tách luồng) và Join (gộp luồng) để thể hiện các tác vụ chạy đồng thời.
- Phân định trách nhiệm (Swimlanes):
  - Chia biểu đồ thành các làn bơi (swimlanes) tương ứng với từng đối tượng, vai trò hoặc actor thực hiện hành động.
- Luồng đối tượng (Object Flow):
  - Thể hiện cách dữ liệu/đối tượng được tạo ra, thay đổi hoặc sử dụng trong suốt quá trình xử lý

Các yếu tố chính trong một biểu đồ hoạt động

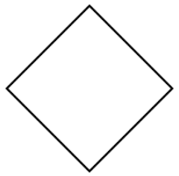
- **Initial Node** khởi đầu:
- **Final Node**: kết thúc
- Activities: bước hoặc hoạt động riêng lẻ trong một quy trình.
- Actions: những hoạt động nguyên tử không thể phân hủy thêm.
- **Control Flow**:Dịch chuyển:
  - Sự chuyển tiếp từ hoạt động này sang hoạt động khác
  - trình tự các hoạt động, được hiển thị bằng cách sử dụng các mũi tên kết nối các hoạt động để minh họa thứ tự thực hiện.
- **Decision Node**:
  - Một điểm mà dòng chảy phân nhánh dựa trên một điều kiện, được biểu thị bằng hình thoi.





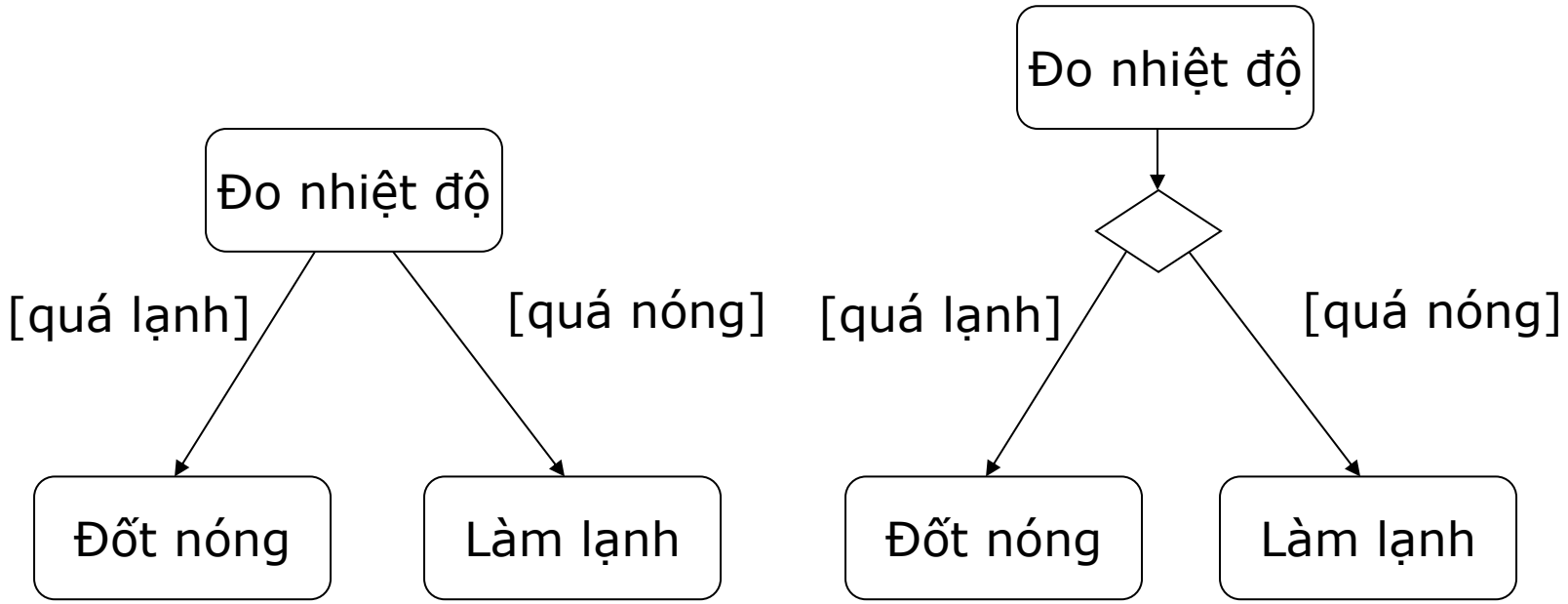






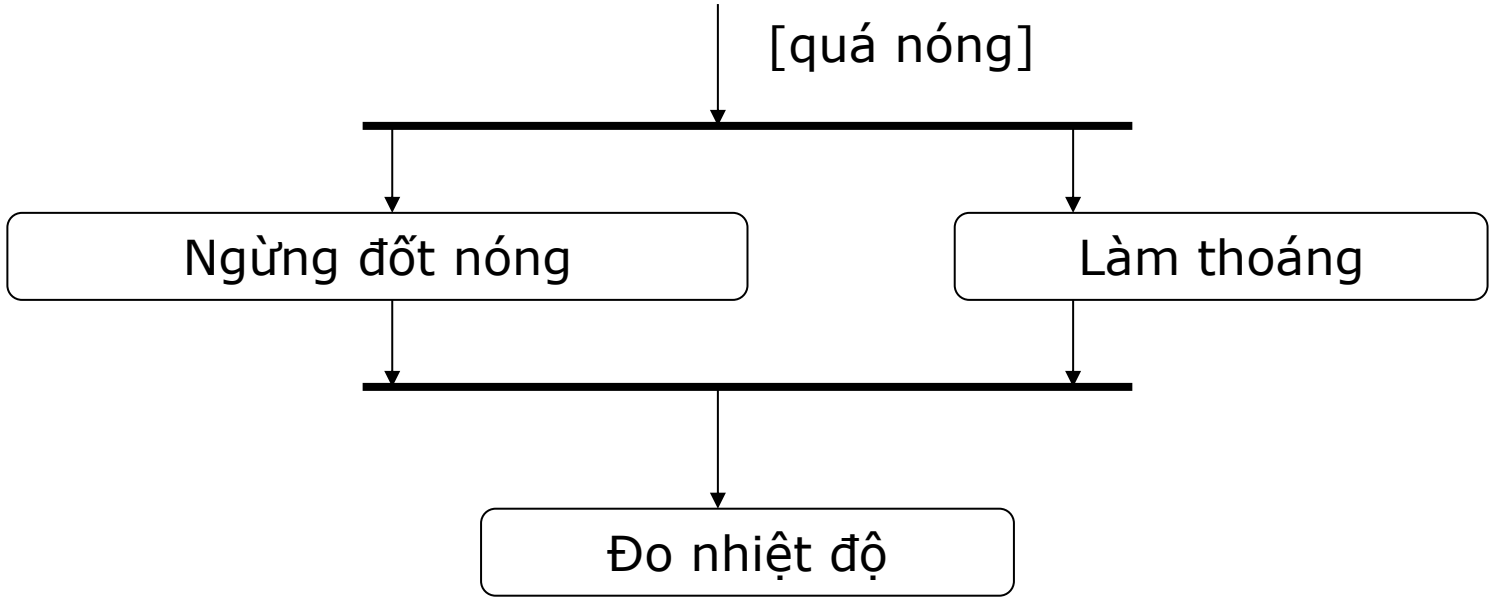
Điều kiện cảnh giới (guard condition)

- Là điều kiện gắn với một dịch chuyển để diễn tả rằng dịch chuyển đó chỉ được thực hiện khi điều kiện này được thoả mãn
- Thường kết hợp với quyết định để thể hiện sự rẽ nhánh



Đồng bộ (synchonization bars)

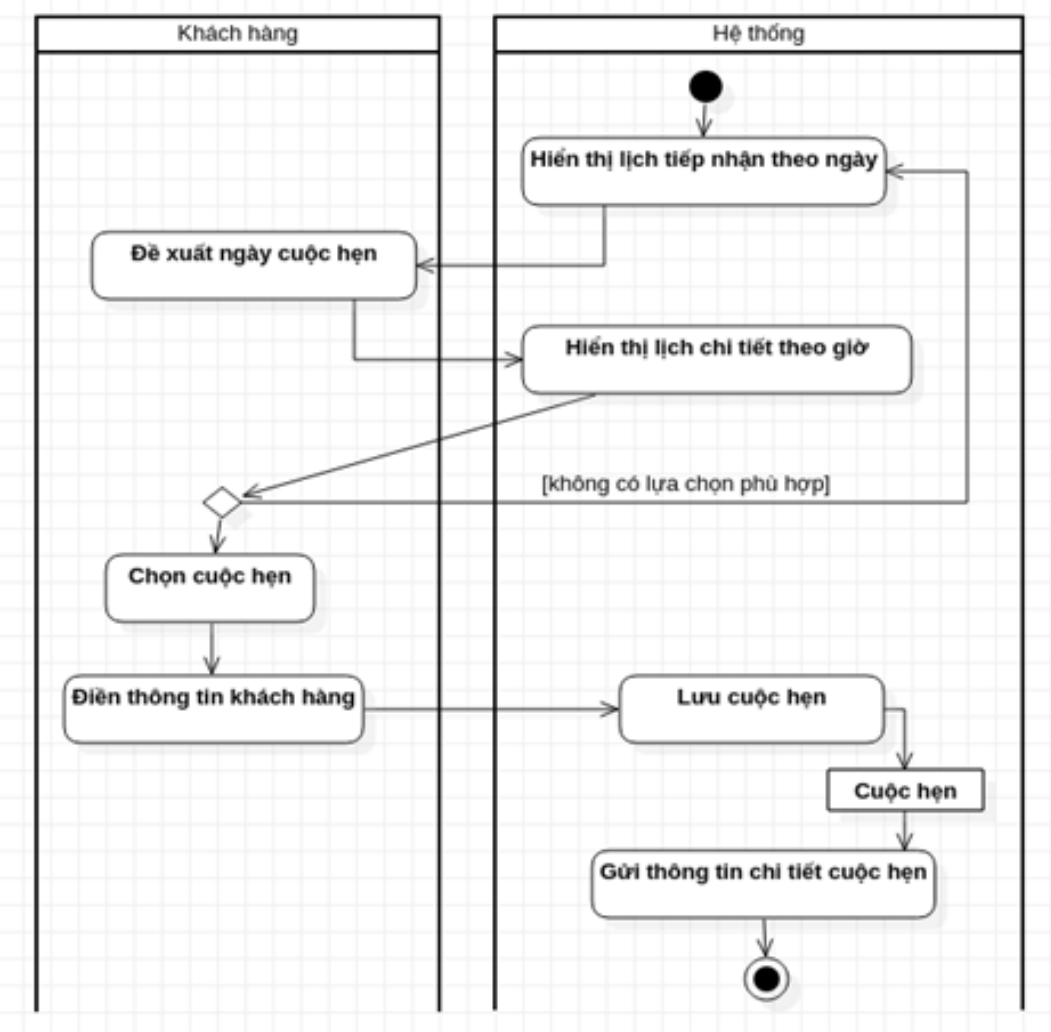
- Dùng để mở hay đóng các nhánh hoạt động song song
- Chạc (fork) : mở các nhánh song song
- Chụm (join) đóng các nhánh song song
- Lưu ý: Khi một nhánh hoạt động đi vào một chạc thì nó phải đi ra từ một chụm



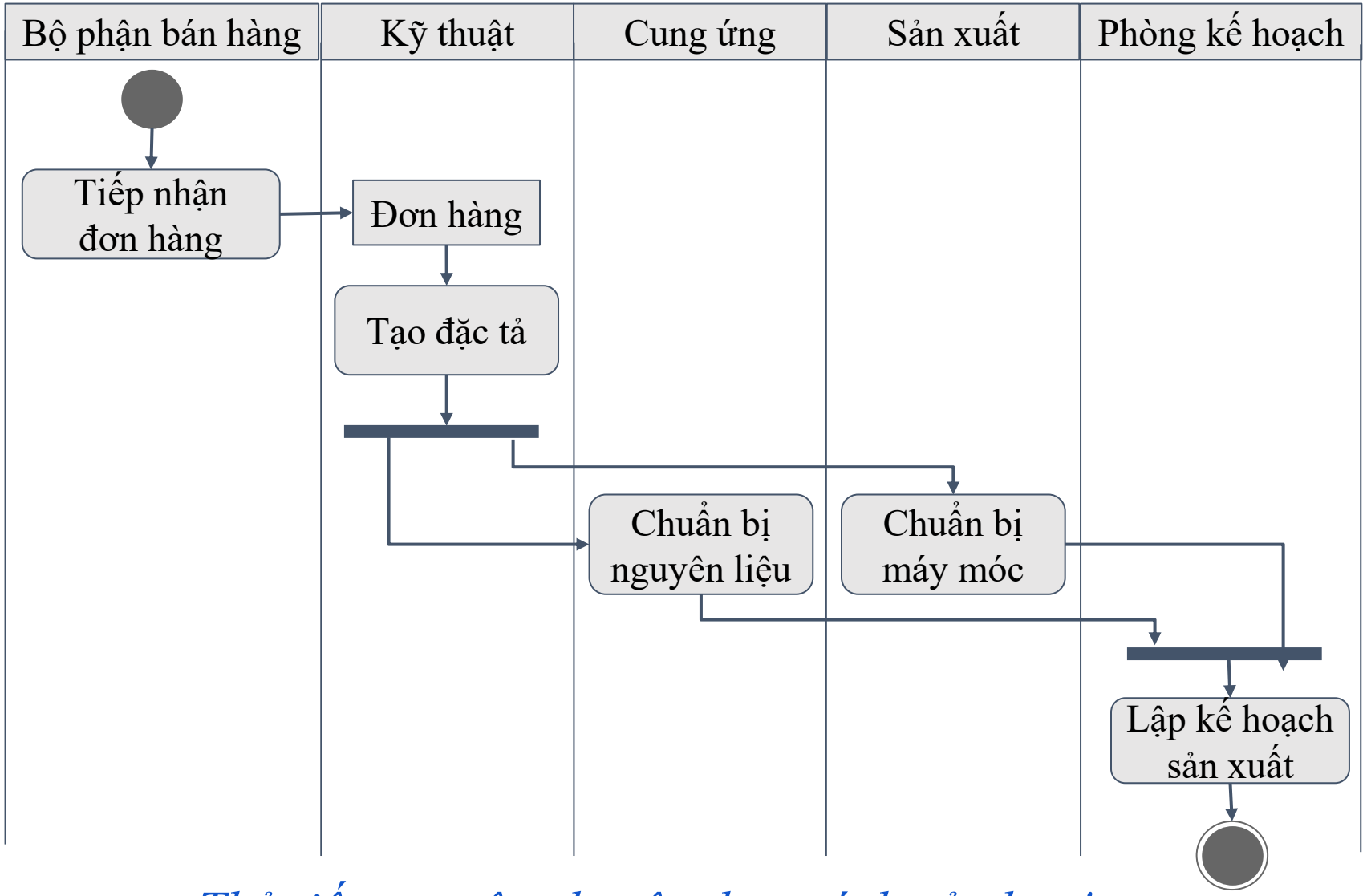
Phân tuyến (swimlane) và phân vùng (pool)

- Dùng để tổ chức các hoạt động theo nhóm, thường đại diện cho các vai trò hoặc trách nhiệm khác nhau.

- Phân vùng: (pool)
  - Phân các hoạt động vào các vùng đại diện cho một hay một nhóm đối tượng liên quan
- Phân tuyến: swimlane
  - Phân các hoạt động vào các tuyến đại diện cho sự phân nhóm của hoạt động



Vd



... Thử tiếp tục câu chuyện theo cách của bạn!

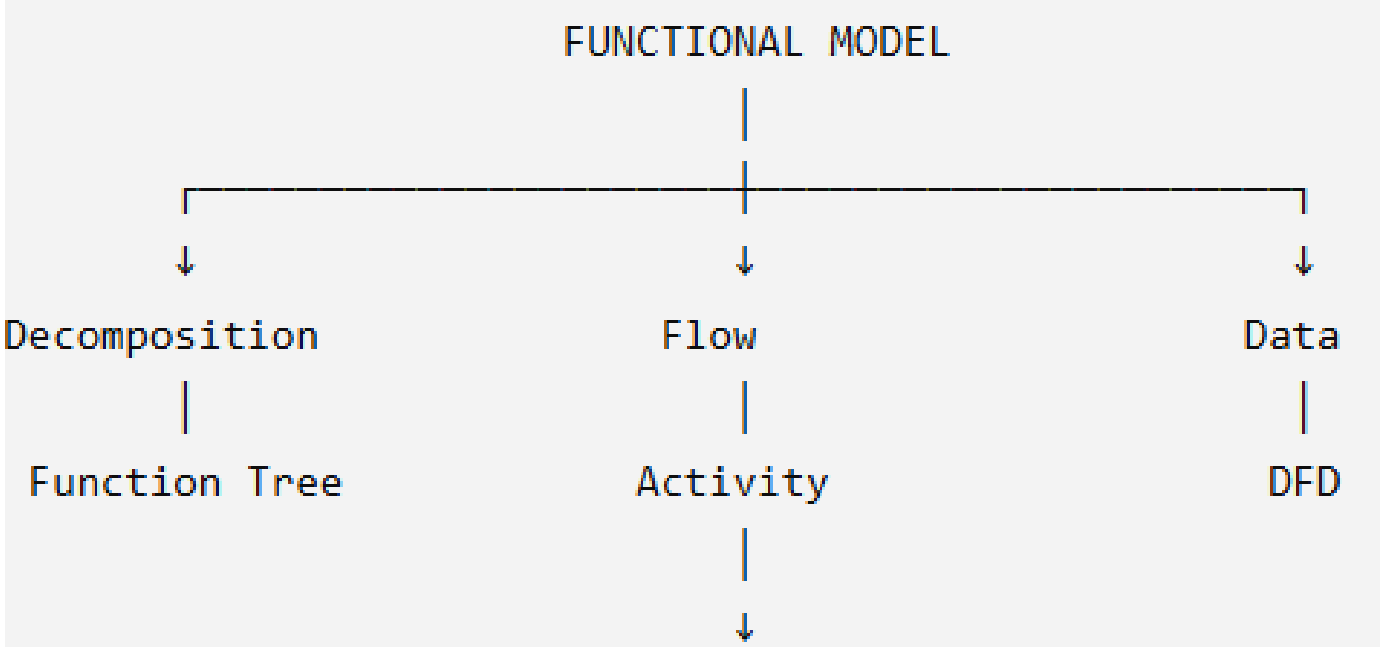
Kết hợp Activity Diagram và BPMN

- Kết hợp theo cấp bậc:
  - Một quy trình lớn có thể được mô tả bằng BPMN, hoạt động chi tiết bên trong một "swim lane" được làm rõ hơn bằng Activity Diagram.
- Kết hợp song song:
  - Sử dụng cả hai sơ đồ để mô tả các khía cạnh khác nhau của cùng một quy trình.
- Sử dụng cho các mục đích khác nhau:
  - Khi cần sự đơn giản và tích hợp UML, sử dụng Activity Diagram.
  - quy trình nghiệp vụ phức tạp, hỗ trợ tự động hóa và tuân thủ chuẩn quốc tế, sử dụng BPMN.

Kỹ thuật Functional modelling khác

Core techniques - Các kỹ thuật Functional Modeling thường dùng

- Core:
  - Functional Decomposition
  - Context Diagram
  - Use Case
  - Functional Flow
  - Activity Diagram
- Nên biết:
  - 6. DFD
    7. Event Modeling
    8. Function–Data mapping
- Mở rộng:
  - 9. Function Point Analysis
    10. Process Specification



6. DFD– nền tảng kinh điển

- Structured Analysis (SA)
  - phương pháp hướng function/process-oriented, trong đó hệ thống được phân rã thành các chức năng/process và mô hình hóa dòng dữ liệu giữa chúng.
- **Function-oriented + Process-oriented + Data-flow-oriented**
  - Structured Analysis nhìn hệ thống theo chức năng và dòng dữ liệu, sau đó phân rã chức năng từ tổng thể xuống các process nhỏ hơn.
- Một hệ thống có thể được hiểu như tập các process biến đổi input thành output thông qua các dòng dữ liệu.

Phân tích Cấu trúc - mô hình hóa chức năng

- Thành phần cơ bản
  - Process / Function – Tiến trình – chức năng
  - Data Flow – Luồng dữ liệu.
  - Data Store – Lưu trữ dữ liệu.
  - External Entity – Thực thể ngoài .
- Nguyên lý Structured Analysis
  - Top-down decomposition
  - Functional decomposition
  - Separation of concerns
  - Hierarchical modeling
  - Data-flow analysis

- Các câu hỏi chính:
  - Hệ thống thực hiện **những process nào?**
  - Dữ liệu **đi vào và đi ra** như thế nào?
  - Process nào tạo ra hoặc biến đổi dữ liệu?
  - Dữ liệu được lưu ở đâu?
  - Các process liên hệ với nhau như thế nào?

- Tư tưởng trung tâm:

SYSTEM

↓

FUNCTIONS / PROCESSES

↓

SUB-PROCESSES

↓

DATA FLOWS

↓

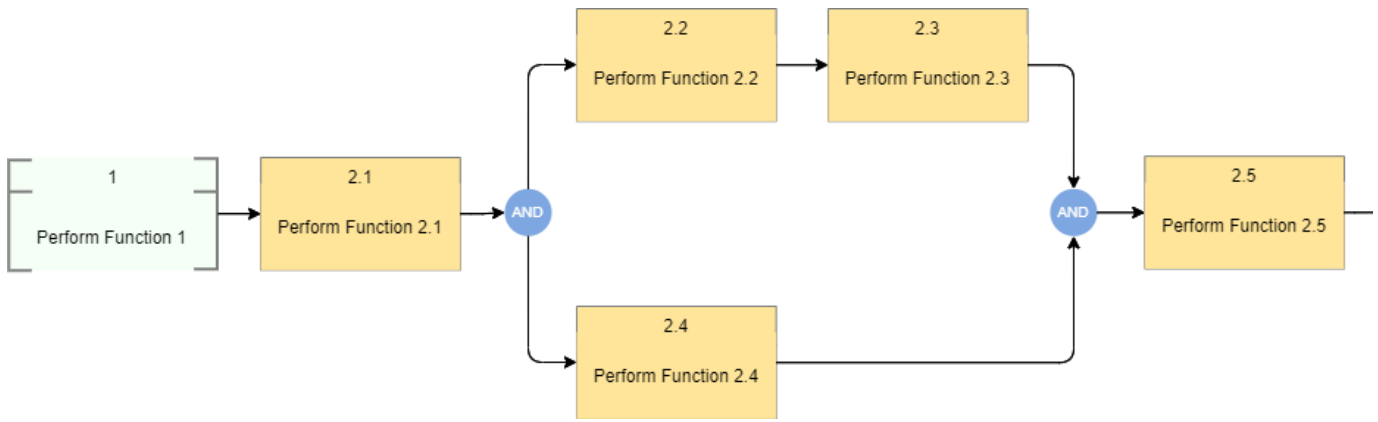
DATA STORES

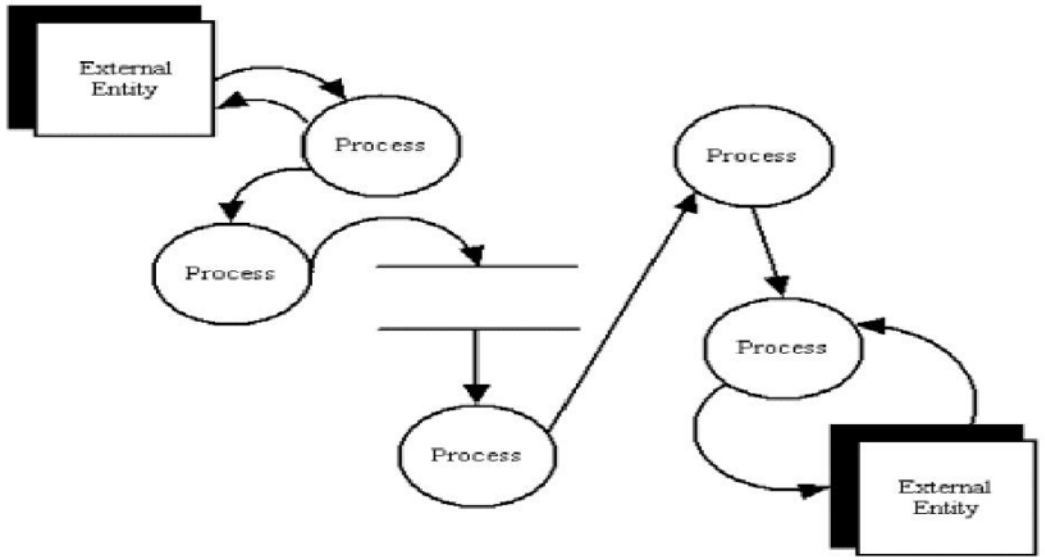
Structured Analysis và Functional Decomposition

- **Điểm quan trọng**
  - Structured Analysis không chỉ hỏi: “Có những chức năng nào?”
  - mà còn :“Các process đó **trao đổi dữ liệu với nhau như thế nào?**”
- Structured Analysis đặc biệt phù hợp với Functional Modeling
  - cung cấp một cách có hệ thống để đi từ **System → Major Functions → Processes → Data Flows**.
- Functional Decomposition có thể được xem là một trong những tư tưởng quan trọng của Structured Analysis.
  - Functional Decomposition tạo cấu trúc chức năng;
  - Structured Analysis mở rộng cấu trúc đó bằng Process và Data Flow.

Bộ công cụ kinh điển của Structured Analysis

- Structured Analysis không phải một biểu đồ duy nhất.
  - Nó sử dụng một nhóm kỹ thuật để mô tả các khía cạnh khác nhau của hệ thống.
  - Trong đó,
    - **DFD là kỹ thuật nổi tiếng nhất**,
    - Functional flow block diagram





7. Event-Driven Functional Modeling

- Không phải mọi function đều được thực hiện vì người dùng nhấn một nút.
- Trong hệ thống doanh nghiệp, nhiều function được kích hoạt bởi sự kiện bên ngoài, thời gian hoặc thay đổi trạng thái.
- Vì vậy Event Modeling
  - giúp mở rộng Functional Modeling từ “user-driven” sang “event-driven”.

- Functional Model phải xét cả **human-initiated functions và event-triggered functions**.

2. Phương pháp tiếp cận dựa trên sự kiện The event-based approach

- là một kỹ thuật toàn diện hơn, tập trung vào kinh doanh.
- coi hệ thống như một "hộp đen" và xác định các sự kiện kinh doanh buộc hệ thống phải phản ứng.

- Nó phân tích sự kiện ở cấp độ **Quy trình nghiệp vụ cơ bản (EBP)**
  - một nhiệm vụ được thực hiện bởi một người ở một địa điểm vào một thời điểm, và kết thúc hệ thống ở trạng thái ổn định

  - Format:
    - Sự kiện : Tên của sự kiện.
    - Nguyên nhân gây ra sự kiện: (ví dụ: tác nhân, thời gian, trạng thái thay đổi).
    - Nguồn/Tác nhân : Ai hoặc cái gì đã khởi xướng sự kiện.
    - Hành động : Phản hồi bắt buộc của hệ thống.
    - Kết quả : Kết quả mong muốn

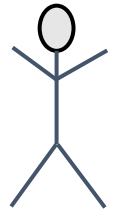
Các loại sự kiện

- External Event Sự kiện bên ngoài:
  - xảy ra bên ngoài phạm vi hệ thống, thường do một tác nhân ngoài khởi xướng
  - (ví dụ: Khách hàng đặt hàng).
- Temporal Event - Sự kiện theo thời gian:
  - xảy ra tự động khi đạt đến một thời điểm cụ thể hoặc một khoảng thời gian nhất định
  - (ví dụ: Cuối tháng sẽ kích hoạt việc tạo báo cáo lương).
- State Event -Sự kiện trạng thái:
  - nội bộ được kích hoạt khi một điều kiện hoặc ngưỡng dữ liệu nhất định bên trong hệ thống được đáp ứng
  - (ví dụ: Lượng hàng tồn kho giảm sẽ kích hoạt thông báo đặt hàng lại tự động)

Sự kiện và ca sử dụng



"Khách hàng thanh toán hóa đơn" - ca sử dụng *Tiếp nhận thanh toán*



"Khách hàng thay đổi địa chỉ" - ca sử dụng *Quản lý hồ sơ khách hàng*



"Khách hàng thực hiện một giao dịch" - ca sử dụng *Xử lý giao dịch*

Sự kiện ngoại phát sinh bên ngoài hệ thống

Sự kiện thời gian và sự kiện trạng thái được kích hoạt bởi các yếu tố bên trong hệ thống

Hệ thống thu phí dịch vụ

"Đến thời gian nhắc nộp phí dịch vụ" - ca sử dụng *Nhắc nộp phí dịch vụ*

"Đến thời gian gửi báo cáo tổng hợp" - ca sử dụng *Gửi báo cáo tổng hợp*

"Gửi cảnh báo nếu lưu lượng sử dụng cao bất thường" - ca sử dụng *Gửi cảnh báo*

Cách thức tiếp cận dựa trên sự kiện

- Xác định các sự kiện kinh doanh:
  - liệt kê và xác định tất cả các sự kiện kinh doanh quan trọng mà hệ thống cần xử lý.
- Xác định hành động cần thiết:
  - với mỗi sự kiện, sẽ xác định phản ứng cần thiết của hệ thống.
- Xác định use-case:
  - Phản hồi của hệ thống đối với các sự kiện trở thành cơ sở để xác định use-case
  - Ví dụ:
    - sự kiện "khách hàng đặt hàng" sẽ dẫn đến ca sử dụng "Xử lý đơn hàng".
- Xác định các tác nhân:
  - các tác nhân kích hoạt hoặc tham gia vào ca sử dụng sẽ được xác định.

Ví dụ: Hệ thống mua sắm trực tuyến

- Bảng sự kiện :
  - Sự kiện bên ngoài : Khách hàng nhấp vào 'Thêm vào giỏ hàng' .
  - Trả lời : Hệ thống thêm sản phẩm vào giỏ hàng.
  - Sự kiện bên ngoài : Khách hàng nhấp vào 'Thanh toán' .
  - Phản hồi : Hệ thống xử lý thanh toán và hoàn tất đơn hàng.
  - Sự kiện tạm thời : Báo cáo hàng tháng phải nộp .
  - Trả lời : Hệ thống tạo báo cáo bán hàng hàng tháng.
- Tác nhân : Khách hàng, Hệ thống, Quản trị viên.
- Các ca sử dụng :
  - Thêm mặt hàng vào giỏ hàng,
  - Xử lý đơn hàng,
  - Tạo báo cáo bán hàng.
- Mối quan hệ :
  - *Process Order* <<*include*>> *Process Payment* use case..
  - *Process Order* <<extent>> failed payment,
- Sơ đồ :

Ưu điểm phương pháp dựa trên sự kiện

- Tính toàn diện cao: Nhận diện đầy yêu cầu do người dùng hay tác nhân tương tác và cả sự kiện thời gian hay trạng thái
- Xác định đúng mức độ phân tích: EBP giúp nhóm công việc tạo ra các Use Case không quá nhỏ hoặc quá lớn.
- Giảm thiểu sót yêu cầu chức năng: tư duy "hệ thống là hộp đen" liệt kê mọi kích hoạt từ môi trường

Nhược điểm

- Độ phức tạp lớn: Dễ gây quá tải thông tin hoặc liệt kê dư thừa khi hệ thống quy mô lớn và phức tạp.
- Yêu cầu kinh nghiệm: cần am hiểu nghiệp vụ kinh doanh để nhận diện chính xác các sự kiện ngầm hoặc sự kiện trạng thái.
- Khó giao tiếp với người dùng không chuyên: trừu tượng so với mục tiêu trực tiếp của người dùng

Function ↔ Data

- Information System luôn xử lý thông tin. Function và Data là hai chiều bổ sung để hiểu Information System.
- Vì vậy một Functional Model hoàn chỉnh cần xem xét function nào tạo, đọc, cập nhật hoặc sử dụng dữ liệu nào.
- Đây là điểm nối quan trọng giữa Functional Modeling và Data/Structural Modeling.

- Điều này giúp phát hiện:
  - Function thiếu data.
  - Data không có function sử dụng.
  - Function tạo dữ liệu nhưng không có nơi lưu.
  - Quyền cập nhật dữ liệu không rõ.

Kiểm tra chất lượng Functional Model

- Một functional model đẹp về hình thức chưa chắc đúng về mặt phân tích.
- Đây là bước quality assurance của Functional Modeling.
  - Trước khi chuyển sang Structural và Behavioral Modeling, cần kiểm tra model có đầy đủ, nhất quán và traceable hay không.
  - Functional Model phải được **validate như một model**, không chỉ kiểm tra syntax của diagram.

Checklist

- **Completeness**
  - Có requirement nào chưa được support?
- **Consistency**
  - Các model có mâu thuẫn không?
- **Correctness**
  - Function có đúng trách nhiệm của system không?
- **Decomposition**
  - Function có được phân rã hợp lý không?
- **Boundary**
  - Có function nào thực ra thuộc external environment?
- **Traceability**
  - Mỗi function quan trọng có requirement/goal tương ứng?
- **Data consistency**
  - Function có input/output hợp lý?

AS-IS → TO-BE → System Functions

Phân tích chức năng bắt đầu từ nghiệp vụ ?

- Lỗi hay gặp nhìn vào một hệ thống rồi liệt kê hàng loạt chức năng.
  - Cách đó có thể tạo ra danh sách chức năng, nhưng không giải thích được **tại sao chức năng đó tồn tại**.
- Functional Modeling cung cấp một cách tiếp cận ngược lại
  - bắt đầu từ **business process**, xác định sự thay đổi mong muốn,
  - rồi mới xác định hệ thống phải làm gì để tạo ra sự thay đổi đó.
- AS-IS → Problems/Gaps → TO-BE → System Functions
  - Quy trình hiện tại đang diễn ra, và quy trình tương lai cần thay đổi thế nào?
  - System Functions được **dẫn xuất từ TO-BE**,
    - dựa trên những hoạt động mà hệ thống cần hỗ trợ để đạt được quy trình tương lai.

System Function cần có nguồn gốc và lý do tồn tại

- Nếu một chức năng xuất hiện trong hệ thống, ta phải trả lời được:
  - Nó giải quyết vấn đề nào?
  - Nó hỗ trợ thay đổi nào trong TO-BE?
  - Nó phục vụ hoạt động nào?
  - Ai cần sử dụng nó?
- System Function
  - kết quả của phân tích và thiết kế quy trình,
  - không phải danh sách chức năng được brainstorm độc lập.

Business Problem

↓

AS-IS Process

↓

Problems / Gaps

↓

TO-BE Process

↓

System-supported Activities

↓

System Functions

Mô hình hóa AS-IS - Quy trình hoạt động ngày nay như thế nào

- Không thể xác định hệ thống mới cần làm gì nếu chưa hiểu hệ thống hiện tại đang giải quyết công việc như thế nào.

- AS-IS không nhằm mô tả "hệ thống cũ" theo nghĩa phần mềm.
  - Nó mô tả business process hiện tại, kể cả những hoạt động hoàn toàn thủ công.
  - Xác định:
    - Actors tham gia
    - Activities đang thực hiện
    - Information được sử dụng
    - Decisions
    - Handoffs
    - Tools / Documents
    - Sequence
    - Các điểm phụ thuộc vào con người

Ví dụ Pet Clinic

Pet Owner gọi điện

→ Receptionist kiểm tra lịch

→ Trao đổi với Doctor

→ Ghi lịch vào Excel

→ Xác nhận lại với Owner

Từ AS-IS đến Problems và Gaps

- Từ AS-IS, đặt câu hỏi:
  - Điều gì đang không hiệu quả hoặc cần thay đổi?
- Đây là bước trung gian rất quan trọng.
  - Nếu bỏ qua Problems/Gaps, nhảy trực tiếp từ AS-IS sang chức năng và thường tạo ra một hệ thống chỉ số hóa quy trình cũ.
- Phân tích Gap giúp biết chính xác điều gì cần thay đổi.
  - Và chính những thay đổi này sẽ là cơ sở để xây dựng TO-BE.

AS-IS	**Problem**
Owner gọi điện	Phụ thuộc Receptionist
Kiểm tra Excel	Chậm, dễ sai
Trao đổi với Doctor	Nhiều handoff
Ghi lịch thủ công	Dễ conflict
Gọi lại Owner	Chậm xác nhận

Xây dựng TO-BE Process

- TO-BE không đơn giản là “AS-IS + máy tính”.
  - AS-IS chỉ cho biết hiện trạng.
  - Để đi tới hệ thống mới, cần phân tích vấn đề và nhu cầu cải thiện.
  - Đây là quy trình nghiệp vụ được thiết kế lại để đạt mục tiêu mong muốn.
- Không thiết kế hệ thống để thay thế công cụ hiện tại;
  - thiết kế hệ thống để giải quyết vấn đề và đạt Business Need.
- Ví dụ,
  - trong AS-IS Receptionist kiểm tra lịch.
  - Trong TO-BE, Owner có thể tự xem lịch.
    - Như vậy không chỉ có một hoạt động được tự động hóa;
    - responsibility đã thay đổi từ con người sang hệ thống.
- Đây chính là điểm bắt đầu để xác định System Functions.

TO-BE = Quy trình nên hoạt động như thế nào

- TO-BE có thể:
  - Remove activity
  - Add activity
  - Automate activity
  - Change responsibility
  - Change sequence
  - Add validation / decision
  - Integrate information

- Ví dụ Pet Clinic:
- Owner
  → View Doctor Availability
  → Select Doctor
  → Select Time Slot
  → System checks availability
  → Book Appointment
  → Confirmation
  → Reminder

AS-IS → TO-BE: Xác định những gì thực sự thay đổi

- Không hỏi:“Có thể thêm chức năng gì?”
  - “Để TO-BE này hoạt động, hệ thống phải làm gì mà AS-IS chưa làm?”
    - System Functions bắt đầu tự xuất hiện từ process .
  - Mỗi thay đổi quan trọng trong TO-BE cần được xem xét xem hệ thống có trách nhiệm gì để tạo ra thay đổi đó.

AS-IS	TO-BE	Change
Gọi Receptionist	Xem lịch trực tuyến	Self-service
Receptionist kiểm tra lịch	System kiểm tra	Automation
Ghi Excel	System lưu booking	Digitalization
Trao đổi thủ công	Chọn Doctor/Slot	Direct interaction
Gọi xác nhận	System confirmation	Automation
Nhắc thủ công	Automatic reminder	Automation

Từ TO-BE → System Responsibility

- TO-BE Activity
  - Xác định ai chịu trách nhiệm →
  - Nếu System chịu trách nhiệm/support → Candidate System Function
  - Không phải mọi hoạt động trong TO-BE đều là System Function.
- xác định system boundary và system responsibility
  - là bước trung gian để tránh nhầm Business Activity = System Function.
  - những phần thuộc System Boundary mới được phân tích thành System Functions.
  - System Responsibility là chiếc cầu nối giữa Business Process và System Function

System Functions → System Use Cases

- System Functions thường là các **capabilities**, chưa nhất thiết là Use Cases hoàn chỉnh.

- TO-BE
  - → System Functions
  - → Group by User Goal
  - → System Use Cases
- Đây là lý do Functional Modeling nên đứng trước Use Case Modeling.
  - System Functions giúp chúng ta biết **system có những capabilities nào**.
  - Use Case Modeling tổ chức các capabilities đó thành **những tương tác có ý nghĩa đối với Actor để đạt một mục tiêu**.
- Như vậy Use Case không phải là nơi "nghĩ ra chức năng", mà là bước **tổ chức và biểu diễn System Functions theo user goal**.

AS-IS → TO-BE → Functional Model

- toàn bộ quá trình là một chuỗi biến đổi
  - từ thực trạng → cải tiến nghiệp vụ → trách nhiệm hệ thống → chức năng hệ thống.
- Functional Modeling
  - không hỏi "phần mềm có những chức năng gì?", mà bắt đầu bằng
  - "business cần thay đổi điều gì và system phải chịu trách nhiệm phần nào?"

AS-IS
↓
Pain Points
↓
Business Needs
↓
Business Goals
↓
TO-BE
↓
System Responsibility
↓
System Functions
↓
Functional Decomposition
↓
Functional Model

TO-BE – Thiết kế cách doanh nghiệp sẽ làm việc

- TO-BE không phải “AS-IS được vẽ đẹp hơn”;
  - cách tổ chức công việc mới dựa trên Business Goals và Business Needs.
- Thay đổi Một số Business Activities:
  - Bị automated.
  - Bị eliminated.
  - Bị simplified.
  - Được reassigned cho System.
  - Vẫn do Human thực hiện.
- Receptionist không còn phải thực hiện toàn bộ các bước thủ công.

System không chỉ automate AS-IS; nó có thể enable TO-BE.

- System không chỉ automate AS-IS; nó có thể enable TO-BE.
  - Mô hình chức năng phải đứng sau Business Modeling + Requirements Modeling, chứ không bằng việc "liệt kê các chức năng của phần mềm".
- Requirements Engineering:

AS-IS → Problem → Need → Goal → TO-BE

↓

Business Requirements

↓

System Requirements

↓

System Functions

Use-Case (BUC -SUC)

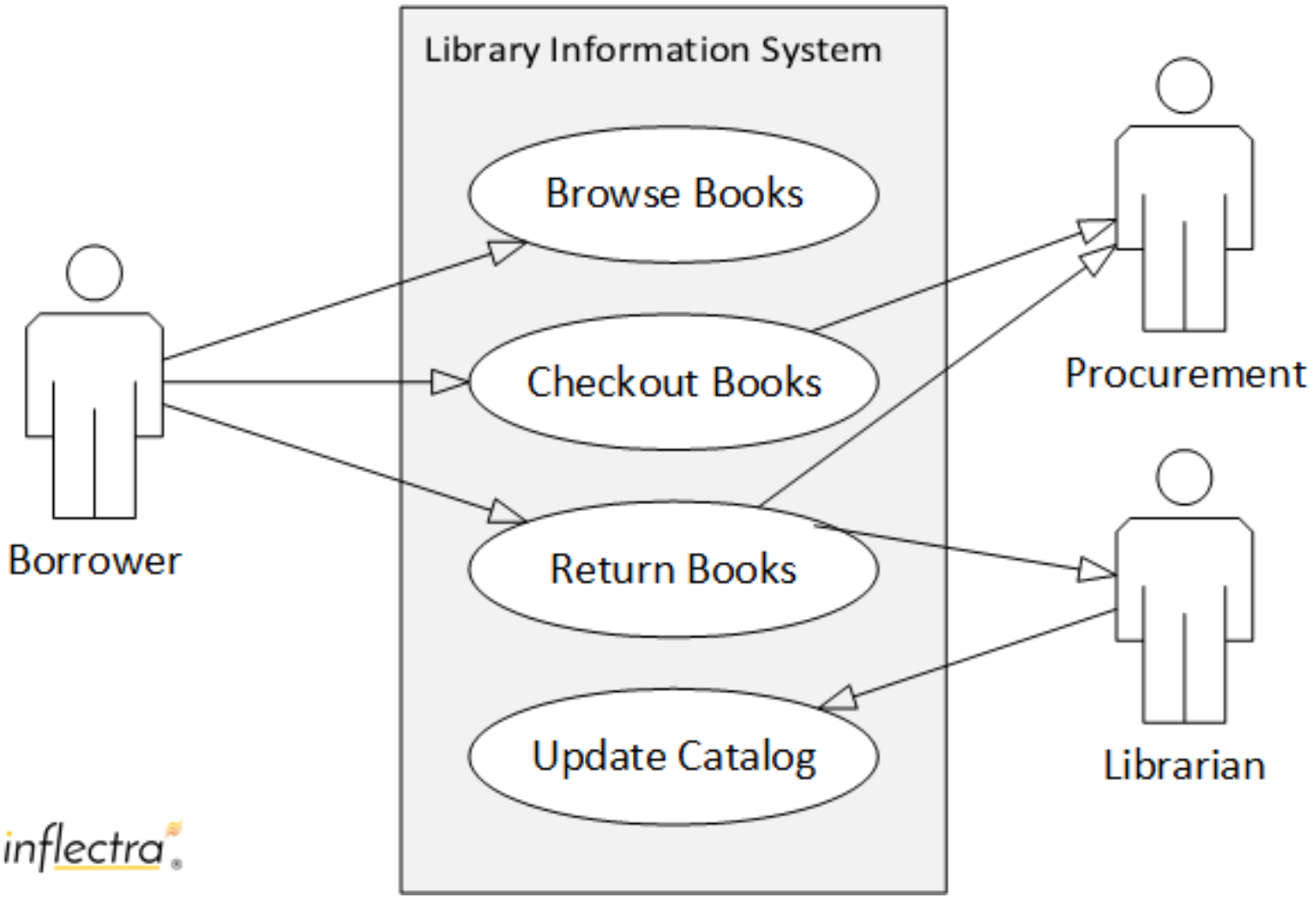
Tại sao BUC vs SUC

- người mới tiếp cận thường nhầm lẫn giữa (BUC) và (SUC) vì chúng đều biểu diễn luồng công việc, nhưng ở hai cấp độ (level) hoàn toàn khác nhau
- Sự khác biệt cốt lõi nằm ở góc nhìn: BUC thuộc về thế giới kinh doanh, còn SUC thuộc về thế giới công nghệ.

- Nguy cơ:
  - Bỏ qua bức tranh lớn:
    - Tập trung ngay vào hệ thống chi tiết (SUC) mà không hiểu rõ mục tiêu doanh nghiệp tổng thể (BUC).
  - Thiếu yêu cầu:
    - Không nắm bắt được các yêu cầu phi chức năng hoặc quy tắc nghiệp vụ quan trọng.
  - Phát triển sai hướng dẫn:
    - Hệ thống được xây dựng không thực sự giải quyết được các vấn đề của doanh nghiệp

Business Use Case (BUC)

- **BUC:** Business User case
  - là một kỹ thuật phổ biến được sử dungđể nắm bắt và ghi lại các yêu cầu chức năng theo quan điểm của người dùng cuối.
- Phương pháp này
  - giúp chuyển đổi các yêu cầu thô, rời rạc từ nhiều nguồn khác nhau (như phỏng vấn, tài liệu và cuộc họp) thành định dạng có cấu trúc bằng sơ đồ hoặc văn bản.
- Mục tiêu của nó là mang lại sự rõ ràng,
  - đảm bảo sự thống nhất giữa các bên liên quan và giảm thiểu rủi ro xây dựng sai sản phẩm.
- **Ví dụ:** BUC
  - Trong hệ thống thư viện:“Quản lý mượn/trả sách” (Borrow/Return Books

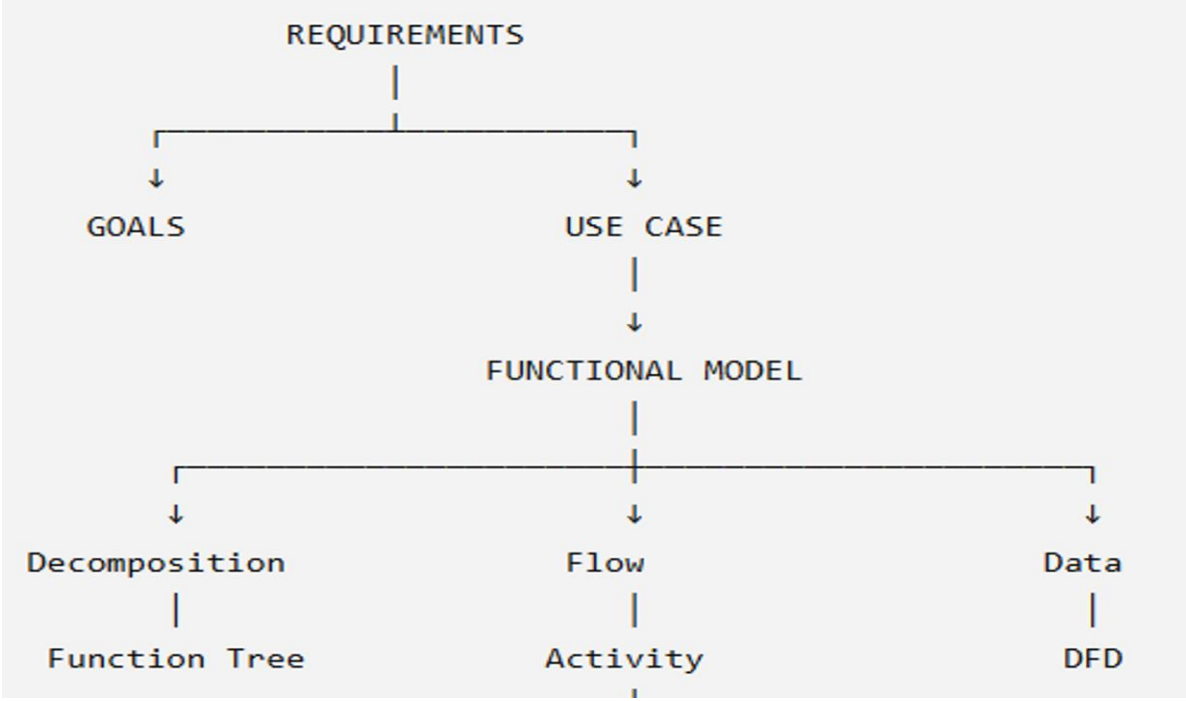

Library Information System
Browse Books
Checkout Books
Return Books
Update Catalog
Borrower
Procurement
Librarian
inflectra

Vd Ứng dụng giao đồ ăn Use Case

- System: Ứng dụng giao đồ ăn di động.
- Primary Actor: Khách hàng đặt bữa ăn.
- Goal: Để đặt hàng và nhận bữa ăn thành công từ nhà hàng.
- Scenario:
  - User browses: lựa chọn nhà hàng và chọn một bữa ăn.
  - User places an order: thông qua ứng dụng.
  - User makes payment: qua ứng dụng hoặc chọn thanh toán khi giao hàng.
  - App sends the order: vào hệ thống nội bộ của nhà hàng.
  - Nhân viên nhà hàng nhận: và xử lý đơn hàng điện tử.
  - Thực phẩm được chuẩn bị, đóng gói và giao hàng: cho khách hàng.

Business User case vs Requirement

- BUC → nguồn gốc của Requirement:
  - Mỗi Business Use Case sinh ra một hoặc nhiều Requirement.
  - Ví dụ:
    - BUC “Quản lý mượn trả sách” → Requirement:
    - Hệ thống phải cho phép kiểm tra tình trạng sách, thông tin mượn, cập nhật số lượng.
- Requirement → cụ thể hóa BUC:
  - Requirement làm rõ cách hệ thống hỗ trợ nghiệp vụ.
  - Từ requirement, người phân tích sẽ tạo System Use Case để đặc tả chi tiết.



System Use Case

- Mô tả các chức năng mà hệ thống phần mềm sẽ thực hiện để hỗ trợ nghiệp vụ.
  - Tập trung vào:
    - How (hệ thống phần mềm thực hiện như thế nào).
    - Các tác nhân hệ thống (system actors: người dùng, hệ thống khác, thiết bị…).
  - Mức độ trừu tượng: Thấp hơn, chi tiết hơn.
  - Ngôn ngữ mô tả: Mang tính kỹ thuật hơn, đủ để lập trình/thiết kế hệ thống.
  - Ví dụ:
    - Trong hệ thống thư viện – “Hệ thống kiểm tra tình trạng sách và ghi nhận thông tin mượn” là một System Use Case.

Business Use Case vs System Use Case

- **Business Use Case (Nghiệp vụ):**
  - Mô tả hoạt động ở mức **tổ chức/kinh doanh**,
  - ví dụ: *Quản lý mượn/trả sách*, *Quản lý đăng ký thành viên*.
  - **Business Use Case** trả lời câu hỏi *"Doanh nghiệp cần làm gì?"*,
- **System Use Case (Hệ thống):**
  - Triển khai chi tiết thành các **tương tác cụ thể của hệ thống**,
  - ví dụ: *Kiểm tra tình trạng sách*, *Ghi nhận thông tin mượn*, *Tạo tài khoản thành viên*.
  - **System Use Case** trả lời *"Hệ thống phải hỗ trợ ra sao?"*.

  - Việc sử dụng cả hai loại use-case này giúp đảm bảo rằng hệ thống phần mềm được xây dựng không chỉ hoạt động tốt về mặt kỹ thuật mà còn thực sự giải quyết được các vấn đề và mục tiêu của doanh nghiệp

Vd Business Use Case (BUC) → Requirement → System Use Case (SUC)

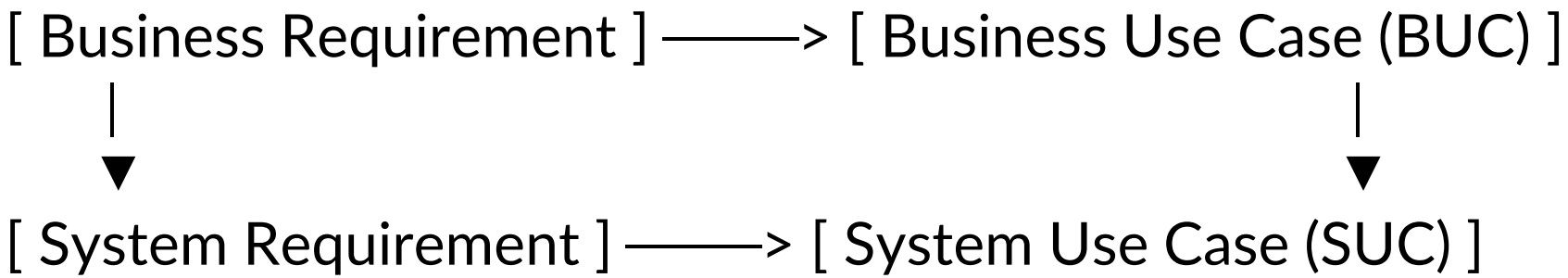
1. Business Use Case (BUC)
   Quản lý mượn/trả sách
   ↓ sinh ra
2. Requirement (Functional & Non-functional)
   "Hệ thống phải cho phép ghi nhận thông tin mượn sách"
   ↓ đặc tả thành
3. System Use Case (SUC)
   "Use Case: Ghi nhận thông tin mượn sách"

- **Business Use Case (BUC) – Mức nghiệp vụ**
  - **Ví dụ:** *"Quản lý mượn/trả sách"* trong thư viện.
  - **Câu hỏi:** Doanh nghiệp cần đạt mục tiêu gì?
  - **Mục tiêu:** Mô tả quy trình nghiệp vụ tổng quan, liên quan đến con người và tổ chức, chưa bàn đến công nghệ.

System Use Case (SUC) – Mức hệ thống

- Hệ thống sẽ tương tác với tác nhân (actor) như thế nào để thực hiện yêu cầu?
  - Đặc tả chi tiết các kịch bản tương tác giữa người dùng ↔ hệ thống,
  - giúp hiểu rõ cần xây dựng gì.
- Ví dụ Yêu cầu - mượn sách:
  - Use Case: Ghi nhận thông tin mượn sách
    - Actor: Thủ thư
    - Pre-condition: Sách phải còn trong kho
    - Steps:
      - Thủ thư nhập mã sách, mã bạn đọc
      - Hệ thống kiểm tra tình trạng sách
      - Nếu hợp lệ → hệ thống ghi nhận giao dịch, cập nhật trạng thái sách
      - Hệ thống in phiếu mượn và lưu log giao dịch

BUC -SUC



- Business Requirements Xác định mục tiêu của tổ chức.
- System Requirements Xác định khả năng chức năng.
- Business Use Cases (BUC)
  - quy trình đáp ứng các yêu cầu kinh doanh.
- System Use Cases (SUC)
  - Ca sử dụng cụ thể đáp ứng các yêu cầu hệ thống

HUST

hust.edu.vn fb.com/dhbkhn